

where f_n is given by (12.1.5).

Up to now we have taken the view that the index n in (12.1.7) varies from $-N/2$ to $N/2$ (cf. 12.1.5). You can easily see, however, that (12.1.7) is periodic in n , with period N . Therefore, $H_{-n} = H_{N-n}$, $n = 1, 2, \dots$. With this conversion in mind, one generally lets the n in H_n vary from 0 to $N - 1$ (one complete period). Then n and k (in h_k) vary exactly over the same range, so the mapping of N numbers into N numbers is manifest. When this convention is followed, you must remember that zero frequency corresponds to $n = 0$ and positive frequencies $0 < f < f_c$ correspond to values $1 \leq n \leq N/2 - 1$, while negative frequencies $-f_c < f < 0$ correspond to $N/2 + 1 \leq n \leq N - 1$. The value $n = N/2$ corresponds to *both* $f = f_c$ and $f = -f_c$.

The discrete Fourier transform has symmetry properties almost exactly the same as the continuous Fourier transform. For example, all the symmetries in the table following equation (12.0.3) hold if we read h_k for $h(t)$, H_n for $H(f)$, and H_{N-n} for $H(-f)$. (Likewise, “even” and “odd” in time refer to whether the values h_k at k and $N - k$ are identical or the negative of each other.)

The formula for the discrete *inverse* Fourier transform, which recovers the set of h_k ’s exactly from the H_n ’s is

$$h_k = \frac{1}{N} \sum_{n=0}^{N-1} H_n e^{-2\pi i k n / N} \quad (12.1.9)$$

Notice that the only differences between (12.1.9) and (12.1.7) are (i) changing the sign in the exponential, and (ii) dividing the answer by N . This means that a routine for calculating discrete Fourier transforms can also, with slight modification, calculate the inverse transforms.

The discrete form of Parseval’s theorem is

$$\sum_{k=0}^{N-1} |h_k|^2 = \frac{1}{N} \sum_{n=0}^{N-1} |H_n|^2 \quad (12.1.10)$$

There are also discrete analogs to the convolution and correlation theorems (equations 12.0.10 and 12.0.12), but we shall defer them to §13.1 and §13.2, respectively.

CITED REFERENCES AND FURTHER READING:

- Brigham, E.O. 1974, *The Fast Fourier Transform* (Englewood Cliffs, NJ: Prentice-Hall).
 James, J.F. 2002, *A Student’s Guide to Fourier Transforms*, 2nd ed. (Cambridge, UK: Cambridge University Press)
 Elliott, D.F., and Rao, K.R. 1982, *Fast Transforms: Algorithms, Analyses, Applications* (New York: Academic Press).

12.2 Fast Fourier Transform (FFT)

How much computation is involved in computing the discrete Fourier transform (12.1.7) of N points? For many years, until the mid-1960s, the standard answer was this: Define W as the complex number

$$W \equiv e^{2\pi i / N} \quad (12.2.1)$$

Then (12.1.7) can be written as

$$H_n = \sum_{k=0}^{N-1} W^{nk} h_k \quad (12.2.2)$$

In other words, the vector of h_k 's is multiplied by a matrix whose (n, k) th element is the constant W to the power $n \times k$. The matrix multiplication produces a vector result whose components are the H_n 's. This matrix multiplication evidently requires N^2 complex multiplications, plus a smaller number of operations to generate the required powers of W . So, the discrete Fourier transform appears to be an $O(N^2)$ process. These appearances are deceiving! The discrete Fourier transform can, in fact, be computed in $O(N \log_2 N)$ operations with an algorithm called the *fast Fourier transform*, or *FFT*. The difference between $N \log_2 N$ and N^2 is immense. With $N = 10^8$, for example, it is a factor of several million, comparable to the ratio of one second to one month. The existence of an FFT algorithm became generally known only in the mid-1960s, from the work of J.W. Cooley and J.W. Tukey. Retrospectively, we now know (see [1]) that efficient methods for computing the DFT had been independently discovered, and in some cases implemented, by as many as a dozen individuals, starting with Gauss in 1805!

One "rediscovery" of the FFT, that of Danielson and Lanczos in 1942, provides one of the clearest derivations of the algorithm. Danielson and Lanczos showed that a discrete Fourier transform of length N can be rewritten as the sum of two discrete Fourier transforms, each of length $N/2$. One of the two is formed from the even-numbered points of the original N , the other from the odd-numbered points. The proof is simply this:

$$\begin{aligned} F_k &= \sum_{j=0}^{N-1} e^{2\pi i j k / N} f_j \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k (2j) / N} f_{2j} + \sum_{j=0}^{N/2-1} e^{2\pi i k (2j+1) / N} f_{2j+1} \\ &= \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j} + W^k \sum_{j=0}^{N/2-1} e^{2\pi i k j / (N/2)} f_{2j+1} \\ &= F_k^e + W^k F_k^o \end{aligned} \quad (12.2.3)$$

In the last line, W is the same complex constant as in (12.2.1), F_k^e denotes the k th component of the Fourier transform of length $N/2$ formed from the even components of the original f_j 's, while F_k^o is the corresponding transform of length $N/2$ formed from the odd components. Notice also that k in the last line of (12.2.3) varies from 0 to N , not just to $N/2$. Nevertheless, the transforms F_k^e and F_k^o are periodic in k with length $N/2$. So each is repeated through two cycles to obtain F_k .

The wonderful thing about the *Danielson-Lanczos lemma* is that it can be used recursively. Having reduced the problem of computing F_k to that of computing F_k^e and F_k^o , we can do the same reduction of F_k^e to the problem of computing the transform of its $N/4$ even-numbered input data and $N/4$ odd-numbered data. In other words, we can define F_k^{ee} and F_k^{eo} to be the discrete Fourier transforms of the

points that are respectively even-even and even-odd on the successive subdivisions of the data.

Although there are ways of treating other cases, by far the easiest case is the one in which the original N is an integer power of 2. In fact, we categorically recommend that you *only* use FFTs with N a power of 2. If the length of your data set is not a power of 2, pad it with zeros up to the next power of 2. (We will give more sophisticated suggestions in subsequent sections below.) With this restriction on N , it is evident that we can continue applying the Danielson-Lanczos lemma until we have subdivided the data all the way down to transforms of length one. What is the Fourier transform of length one? It is just the identity operation that copies its one input number into its one output slot! In other words, for every pattern of $\log_2 N$ e 's and o 's, there is a one-point transform that is just one of the input numbers f_n ,

$$F_k^{eooooo\cdots oee} = f_n \quad \text{for some } n \quad (12.2.4)$$

(Of course this one-point transform actually does not depend on k , since it is periodic in k with period 1.)

The next trick is to figure out which value of n corresponds to which pattern of e 's and o 's in equation (12.2.4). The answer is: Reverse the pattern of e 's and o 's, then let $e = 0$ and $o = 1$, and you will have, *in binary*, the value of n . Do you see why it works? It is because the successive subdivisions of the data into even and odd are tests of successive low-order (least significant) bits of n . This idea of *bit reversal* can be exploited in a very clever way that, along with the Danielson-Lanczos lemma, makes FFTs practical: Suppose we take the original vector of data f_j and rearrange it into bit-reversed order (see Figure 12.2.1), so that the individual numbers are in the order not of j , but of the number obtained by bit reversing j . Then the bookkeeping on the recursive application of the Danielson-Lanczos lemma becomes extraordinarily simple. The points as given are the one-point transforms. We combine adjacent pairs to get two-point transforms, then combine adjacent pairs of pairs to get four-point transforms, and so on, until the first and second halves of the whole data set are combined into the final transform. Each combination takes of order N operations, and there are evidently $\log_2 N$ combinations, so the whole algorithm is of order $N \log_2 N$ (assuming, as is the case, that the process of sorting into bit-reversed order is no greater in order than $N \log_2 N$).

This, then, is the structure of an FFT algorithm: It has two sections. The first section sorts the data into bit-reversed order. Luckily this takes no additional storage, since it involves only swapping pairs of elements. (If k_1 is the bit reverse of k_2 , then k_2 is the bit reverse of k_1 .) The second section has an outer loop that is executed $\log_2 N$ times and calculates, in turn, transforms of length 2, 4, 8, $\dots$, N . This series of operations is often called a *butterfly*. For each stage of the process, two nested inner loops range over the subtransforms already computed and the elements of each transform, implementing the Danielson-Lanczos lemma. The operation is made more efficient by restricting external calls for trigonometric sines and cosines to the outer loop, where they are made only $\log_2 N$ times. Computation of the sines and cosines of multiple angles is through simple recurrence relations in the inner loops (cf. 5.4.6).

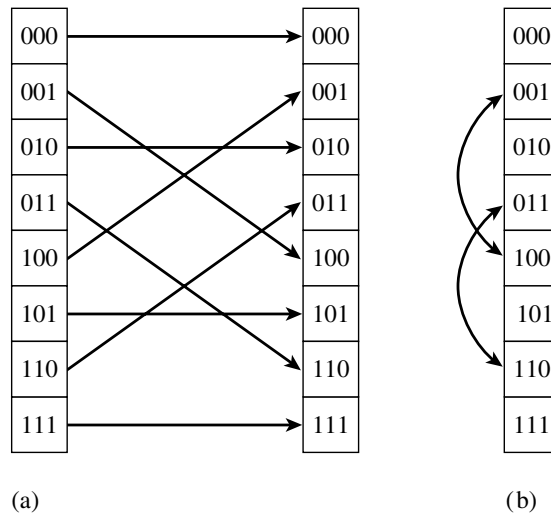


Figure 12.2.1. Reordering an array (here of length 8) by bit reversal, (a) between two arrays, versus (b) in place. Bit-reversal reordering is a necessary part of the fast Fourier transform (FFT) algorithm.

12.2.1 Bare FFT Routine and Helper Interfaces

Experience convinces us that a good way to package the FFT is as (i) a bare routine with a minimal interface, plus also (ii) a small set of interface objects that make it easier to get data in and out of the bare routine. The bare FFT routine given below is based on one originally written by N.M. Brenner. The input quantities are the number of complex data points n ($=N$), a pointer to the data array (`data[0..2*n-1]`), and `isign`, which is set to either ± 1 and is the sign of i in the exponential of equation (12.1.7). When `isign` is set to -1 , the routine thus calculates the inverse transform (12.1.9) — except that it does not multiply by the normalizing factor $1/N$ that appears in that equation. You do that yourself. We test to be sure that n is a power of 2 by the C++ idiom `n&(n-1)`, which is zero only if n is, in binary, 1 followed by any number of zeros.

Notice that the argument n is the number of *complex* data points. The actual length of the Doub array (`data[0..2*n-1]`) is $2n$, with each complex value occupying two consecutive locations. In other words, `data[0]` is the real part of f_0 , `data[1]` is the imaginary part of f_0 , and so on up to `data[2*n-2]`, which is the real part of f_{N-1} , and `data[2*n-1]`, which is the imaginary part of f_{N-1} .

The FFT routine gives back the F_n 's packed in exactly the same fashion, as n complex numbers. The real and imaginary parts of the zero frequency component F_0 are in `data[0]` and `data[1]`; the smallest nonzero positive frequency has real and imaginary parts in `data[2]` and `data[3]`; the smallest (in magnitude) nonzero negative frequency has real and imaginary parts in `data[2*n-2]` and `data[2*n-1]`. Positive frequencies increasing in magnitude are stored in the real-imaginary pairs `data[4]`, `data[5]` up to `data[n-2]`, `data[n-1]`. Negative frequencies of increasing magnitude are stored in `data[2*n-4]`, `data[2*n-3]` down to `data[n+2]`, `data[n+3]`. Finally, the pair `data[n]`, `data[n+1]` contains the real and imaginary parts of the one aliased point that contains the most positive and the most negative frequencies. You should try to develop a familiarity with this storage arrangement of

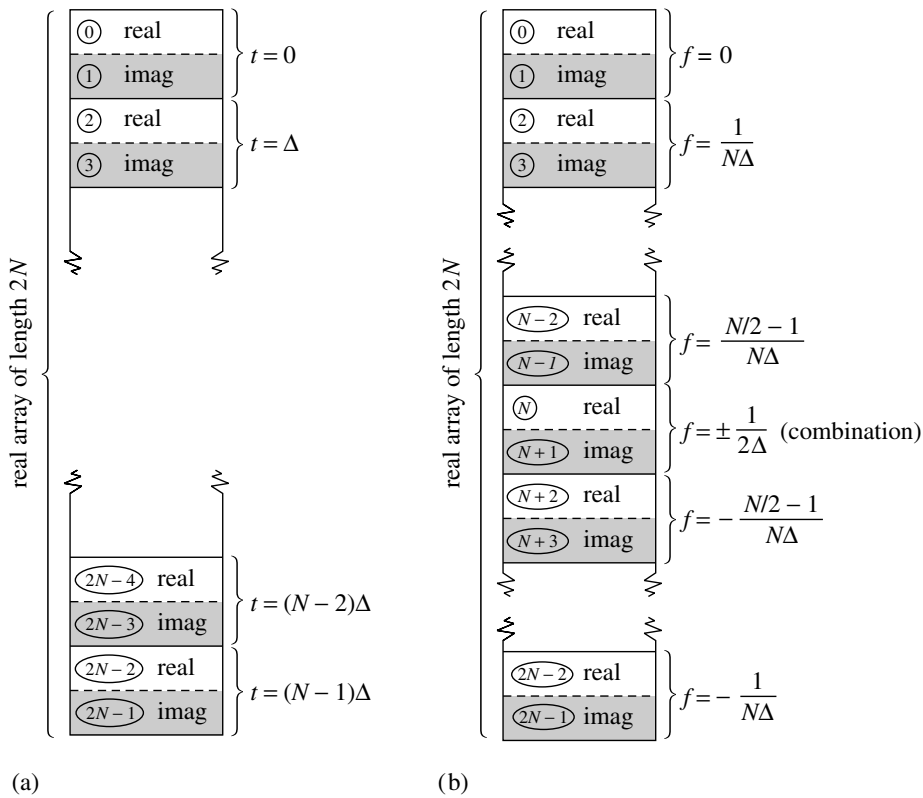


Figure 12.2.2. Input and output arrays for FFT. (a) The input array contains N (a power of 2) complex time samples in a real array of length $2N$, with real and imaginary parts alternating. (b) The output array contains the complex Fourier spectrum at N values of frequency. Real and imaginary parts again alternate. The array starts with zero frequency, works up to the most positive frequency (which is aliased with the most negative frequency). Negative frequencies follow, from the second-most negative up to the frequency just below zero.

complex spectra, also shown in Figure 12.2.2, since it is the practical standard.

```
fourier.h void four1(Doub *data, const Int n, const Int isign) {
    Replaces data[0..2*n-1] by its discrete Fourier transform, if isign is input as 1; or replaces
    data[0..2*n-1] by n times its inverse discrete Fourier transform, if isign is input as -1. data
    is a complex array of length n stored as a real array of length 2*n. n must be an integer power
    of 2.
    Int nn, mmax, m, j, istep, i;
    Doub wtemp, wr, wpr, wpi, wi, theta, tempr, tempi;
    if (n<2 || n&(n-1)) throw("n must be power of 2 in four1");
    nn = n << 1;
    j = 1;
    for (i=1; i<nn; i+=2) {
        if (j > i) {
            SWAP(data[j-1], data[i-1]);
            SWAP(data[j], data[i]);
        }
        m=n;
        while (m >= 2 && j > m) {
            j -= m;
            m = m/2;
        }
        theta = 2 * PI * j / m;
        wtemp = cos(theta);
        wpr = 1;
        wpi = 0;
        while (m >= 2) {
            wpr = wtemp * wpr - wpi * wpi;
            wpi = wtemp * wpi + wpr * wpr;
            wtemp = wpr;
            wpr = 1;
            wpi = 0;
            m = m/2;
        }
        istep = m/2;
        while (istep > 0) {
            for (m=istep; m<nn; m+=2*istep) {
                for (i=m; i<nn; i+=2) {
                    j = i + istep;
                    SWAP(data[j-1], data[i-1]);
                    SWAP(data[j], data[i]);
                }
            }
            istep = istep/2;
        }
    }
}
```

This is the bit-reversal section of the routine.
Exchange the two complex numbers.

```

        m >>= 1;
    }
    j += m;
}

```

Here begins the Danielson-Lanczos section of the routine.

```

mmax=2;
while (nn > mmax) {
    istep=mmax << 1;
    theta=isign*(6.28318530717959/mmax);
    wtemp=sin(0.5*theta);
    wpr = -2.0*wtemp*wtemp;
    wpi=sin(theta);
    wr=1.0;
    wi=0.0;
    for (m=1;m<mmax;m+=2) {
        for (i=m;i<=nn;i+=istep) {
            j=i+mmax;
            tempr=wr*data[j-1]-wi*data[j];
            tempi=wr*data[j]+wi*data[j-1];
            data[j-1]=data[i-1]-tempr;
            data[j]=data[i]-tempi;
            data[i-1] += tempr;
            data[i] += tempi;
        }
        wr=(wtemp*wr)*wpr-wi*wpi+wr;
        wi=wi*wpr+wtemp*wpi+wi;
    }
    mmax=istep;
}

```

Outer loop executed $\log_2 n$ times.

Initialize the trigonometric recurrence.

Here are the two nested inner loops.

This is the Danielson-Lanczos formula:

Trigonometric recurrence.

For an interface at a slightly higher level, we can overload the bare `four1` with equivalent functions that input and output data as either a `VecDoub` of length $2N$ or a `VecComplex` of length N :

```

void four1(VecDoub_IO &data, const Int isign) {
    Overloaded interface to four1. Replaces the vector data, a complex vector of length  $N$  stored
    as a real vector of twice that length, by its discrete Fourier transform, with components in
    wraparound order, if isign is 1; or by  $N$  times the inverse Fourier transform, if isign is -1.
    four1(&data[0],data.size()/2,isign);
}

void four1(VecComplex_IO &data, const Int isign) {
    Overloaded interface to four1. Replaces the vector data, a complex vector of length  $N$  stored
    as such, by its discrete Fourier transform, with components in wraparound order, if isign is 1;
    or by  $N$  times the inverse Fourier transform, if isign is -1.
    four1((Doub*)&data[0],data.size(),isign);
}

```

fourier.h

In these overloaded versions, however, you are still responsible for decoding on your own the wraparound order. To get an interface that takes care of that for you, we can define an object `WrapVecDoub` that creates a real vector (or binds a reference to an existing one) and then defines methods for addressing the vector as if it were a complex vector of half the size. Since the `WrapVecDoub` object also knows about wraparound periodicity, you can access frequencies either by subscripts $[0 \dots n-1]$ or $[-n/2 \dots n/2-1]$, or, for that matter, any n consecutive complex components. The object has a conversion operator to `VecDoub`, so you can (e.g.) send it directly to `four1`.

fourier.h

```
struct WrapVecDoub {
```

Object for accessing a VecDoub as if it were a complex vector of half the length, with wraparound periodicity.

```
    VecDoub vvec;                Used when data are stored internally.
    VecDoub &v;
    Int n, mask;
```

```
    WrapVecDoub(const Int nn) : vvec(nn), v(vvec), n(nn/2),
    mask(n-1) {validate();}
```

Constructor. Declare a new vector with *nn* real components (half as many complex).

```
    WrapVecDoub(VecDoub &vec) : v(vec), n(vec.size()/2),
    mask(n-1) {validate();}
```

Constructor. Bind the data in an existing vector *vec* for access as if complex.

```
    void validate() {if (n&(n-1)) throw("vec size must be power of 2");}
```

```
    inline Complex& operator[] (Int i) {return (Complex &)v[(i&mask) << 1];}
```

Reduce any integer *i* to the periodic range $[0..n]$ and return that complex component. Can also be an l-value.

```
    inline Doub& real(Int i) {return v[(i&mask) << 1];}
```

As above, but return the real part only. Can also be an l-value.

```
    inline Doub& imag(Int i) {return v[(i&mask) << 1]+1];}
```

As above, but return the imaginary part only. Can also be an l-value.

```
    operator VecDoub&() {return v;}
```

Conversion operator. Allows a WrapVecDoub object to be sent to any function that expects a VecDoub.

```
};
```

Here are some sample lines of code (not a useful program) that show how WrapVecDoub can be used:

```
Int j,n=256;
VecDoub dat(2*n);
WrapVecDoub data1(dat), data2(2*n);
for (j=0;j<n;j++) {
    data1[j] = Complex(... , ...);
    data2.real(j) = ... ;
    data2.imag(j) = ... ;
}
four1(data1,1);
four1(data2,1);
for (j=-n/2;j<n/2;j++) {
    ... = data1.real(j);
    ... = data2[j];
}
```

256 complex components, e.g.
A real vector to hold them.
Examples of the two constructors.
Loop over complex components.
Set a complex value directly,
or set real and imag separately.

Invokes four1(VecDoub&,Int) through the
conversion operator.
Can address negative frequencies directly!
Get real part of component *j*.
Get component as a complex value.

12.2.2 Decomposing the FFT for Parallelism

It is possible to decompose the calculation of an FFT of size N into a set of smaller FFTs that can be done independently of one another. This can be useful either to achieve parallelism or to allow more versatile memory management. The basic idea is to address the input array as if it were a two-dimensional array of size $m \times M$, where $N = mM$ and N , m , and M are all powers of 2. Then the components of f can be addressed as

$$f[Jm + j], \quad 0 \leq j < m, \quad 0 \leq J < M \quad (12.2.5)$$

where the j index changes more rapidly, the J index more slowly, and brackets denote C++ subscripts.

What we want to compute is the FFT of the original array of length N , which we can write as

$$F[kM + K] \equiv \sum_{j,J} e^{2\pi i(kM+K)(Jm+j)/(Mm)} f(Jm + j), \quad (12.2.6)$$

$$0 \leq k < m, \quad 0 \leq K < M$$

You can see that the indices k and K address the desired result (FFT of the original array), with K varying more rapidly.

From equation (12.2.6) it is easy to verify the identity

$$F[kM + K] = \sum_j \left\{ e^{2\pi i j k / m} \left[e^{2\pi i j K / (Mm)} \left(\sum_J e^{2\pi i J K / M} f(Jm + j) \right) \right] \right\} \quad (12.2.7)$$

But this, reading it from the innermost operation outward, is just the method that we need:

- For each value of j form a vector of input values whose components vary with J , $0 \leq J < M$. This is basically a transpose operation.
- FFT each such vector. (OK to do in parallel.) Notationally, the J index now becomes a K index.
- Multiply each component by a phase factor $\exp[2\pi i j K / (Mm)]$.
- Rearrange the data so that they are accessible as a set of vectors whose components vary with j , $0 \leq j < m$, another transpose operation.
- FFT each such vector. (OK to do in parallel.) The j index now becomes a k index.
- The answer is now available as $F[kM + K]$. It takes a third transpose operation to get it back into the desired order (with k varying most rapidly).

Even though two FFTs are performed on each element, the operations count is about the same as for the ordinary FFT: The first set of FFTs scales as $N \log M$, the second set as $N \log m$, and the total is thus $N \log(Mn) = N \log N$.

For further discussion, see [2], where the above is called the *six-step framework*. You can easily eliminate the first two of the three transpose operations by writing a new `four1` routine with an additional *stride* argument, specifying the constant increment between logically “next” components. The stride will be m for the first set of FFTs, and 1 for the second set. An algorithm very similar to this is called the *four-step framework*. See [2,3] for more details.

Related decompositions, called *zoom transforms*, can be used to get an approximation to the spectrum of a long data stream, at high resolution in only certain frequency bands. See [4-6].

12.2.3 Other FFT Algorithms

We should mention that there are a number of variants on the basic FFT algorithm given above. As we have seen, that algorithm first rearranges the input elements into bit-reverse order, then builds up the output transform in $\log_2 N$ iterations. In the literature, this sequence is called a *decimation-in-time* or *Cooley-Tukey* FFT

algorithm. It is also possible to derive FFT algorithms that first go through a set of $\log_2 N$ iterations on the input data, and rearrange the *output* values into bit-reverse order. These are called *decimation-in-frequency* or *Sande-Tukey* FFT algorithms. For some applications, such as convolution (§13.1), one takes a data set into the Fourier domain and then, after some manipulation, back out again. In these cases it is possible to avoid all bit reversing. You use a decimation-in-frequency algorithm (without its bit reversing) to get into the “scrambled” Fourier domain, do your operations there, and then use an inverse algorithm (without *its* bit reversing) to get back to the time domain. While elegant in principle, this procedure does not in practice save much computation time, since the bit reversals represent only a small fraction of an FFT’s operations count, and since most useful operations in the frequency domain require a knowledge of which points correspond to which frequencies.

Another class of FFTs subdivides the initial data set of length N not all the way down to the trivial transform of length 1, but rather only down to some other small power of 2, for example $N = 4$, *base-4 FFTs*, or $N = 8$, *base-8 FFTs*. These small transforms are then done by small sections of highly optimized coding that take advantage of special symmetries of that particular small N . For example, for $N = 4$, the trigonometric sines and cosines that enter are all ± 1 or 0, so many multiplications are eliminated, leaving largely additions and subtractions. These can be faster than simpler FFTs by some significant, but not overwhelming, factor, e.g., 20 or 30%.

There are also FFT algorithms for data sets of length N not a power of 2. They work by using relations analogous to the Danielson-Lanczos lemma to subdivide the initial problem into successively smaller problems, not by factors of 2, but by whatever small prime factors happen to divide N . The larger that the largest prime factor of N is, the worse this method works. If N is prime, then no subdivision is possible, and the user (whether he knows it or not) is taking a *slow* Fourier transform, of order N^2 instead of order $N \log_2 N$. Our advice is to stay clear of such FFT implementations, with perhaps one class of exceptions, the *Winograd Fourier transform algorithms*. Winograd algorithms are in some ways analogous to the base-4 and base-8 FFTs. Winograd has derived highly optimized codings for taking small- N discrete Fourier transforms, e.g., for $N = 2, 3, 4, 5, 7, 8, 11, 13, 16$. The algorithms also use a different and clever way of combining the subfactors. The method involves a reordering of the data both before the hierarchical processing and after it, but it allows a significant reduction in the number of multiplications in the algorithm. For some especially favorable values of N , the Winograd algorithms can be significantly (e.g., up to a factor of 2) faster than the simpler FFT algorithms of the nearest integer power of 2. This advantage in speed, however, must be weighed against the considerably more complicated data indexing involved in these transforms, and the fact that the Winograd transform cannot be done “in place.”

Finally, an interesting class of transforms for doing convolutions quickly is number-theoretic transforms [7,8]. These schemes replace floating-point arithmetic with integer arithmetic modulo some large prime $N+1$, and the N th root of 1 by the modulo arithmetic equivalent. Strictly speaking, these are not *Fourier* transforms at all, but the properties are quite similar and computational speed can be far superior. On the other hand, their use is somewhat restricted to quantities like correlations and convolutions since the transform itself is not easily interpretable as a “frequency” spectrum.